

Master's thesis
Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 9, 2026

Supervisor(s): Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Examiner(s): Keijo Heljanko

University of Helsinki
Faculty of Science

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Master's thesis	August 9, 2026	21

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS):
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — Övriga uppgifter — Additional information

Software study track

Contents

1 Introduction **1**

2 Background: Systems Languages for Latency-Critical Software **3**

3 Problem Context: UE Context Management and Predictability **5**

4 Objectives **7**

5 Research Methodology **9**

6 Design and Implementation **11**

7 Experimental Setup and Evaluation **13**

8 Discussion **15**

9 Conclusion and Future Work **17**

Bibliography **19**

Appendix A Fault-injection example **21**

1. Introduction

- The 5G RAN control plane (the gNB-CU-CP) now runs as software on ordinary servers. It is large, it keeps state for every connected device, it serves many requests at once, and it has only a few milliseconds to complete each procedure [1]. One defect, or one regression in the slowest requests, therefore degrades the whole site.
- This software is written mostly in C++. C++ gives the control over memory and timing that such a system needs, but it leaves memory-safety and concurrency mistakes to the programmer [2]. Rust promises the same performance while rejecting those mistakes at compile time [3].
- The problem is a lack of evidence. Telecom vendors are under growing pressure to move to memory-safe languages [4], but no controlled study tells them what the change would mean for predictability, how many defects it would really prevent, or what the migration would cost for a representative control-plane subsystem [5].
- Four research questions frame the work. RQ1 compares tail latency and jitter under overload [6]. RQ2 asks which defect classes Rust removes at compile time. RQ3 asks what Rust's safety model costs in resources. RQ4 asks how expensive it is to adopt Rust step by step inside an existing C++ codebase.
- The work contributes four things: a benchmarking framework that others can rerun, one frozen specification implemented twice and tested for identical behaviour, a defect taxonomy that also marks where Rust's guarantee ends, and migration guidelines drawn from the measurements.

2. Background: Systems Languages for Latency-Critical Software

- In the NG-RAN architecture the base station (gNB) is split into a central and a distributed unit, and the central unit is split again into a control plane and a user plane. UE Context Management sits in the control plane, the gNB-CU-CP, and reaches its neighbours over the standardised F1, E1, and NG interfaces [7].
- Deadlines are met or missed because of predictability, not average speed. The metrics that matter are therefore the p99 and p99.9 latencies and the jitter around them [6]. They have to be measured past the saturation point and recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests [8].
- C++ offers abstractions that cost nothing at run time, deterministic cleanup through RAII, and no garbage collector. Safety, however, remains the programmer's responsibility [2], and sanitizers report only the errors that a given test run happens to reach.
- Rust has the same run-time model and also no garbage collector, but it checks memory safety and freedom from data races at compile time, using ownership, borrowing, and the `Send`/`Sync` traits [9]. Code that needs to step outside those rules must be marked `unsafe`, which keeps such code small and easy to find.
- The comparison is therefore C++ against Rust. Garbage-collected languages are left out because their pauses are incompatible with the latency targets. The concurrency model is held fixed in both implementations, and the C ABI is treated as the realistic adoption path. No earlier study isolates the language this way [5].

3. Problem Context: UE Context Management and Predictability

- UE Context Management owns the lifecycle of one context per connected device. Each context moves through four states, Idle, Connected, Reconfiguring, and Released, driven by a small set of lifecycle operations. An operation that is illegal in the current state returns a typed error instead of being carried out.
- The subsystem was chosen because it is both representative and small enough to keep under control. It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets [1], and it is still simple enough to specify precisely and to test for identical behaviour in two languages.
- Five defect classes fall inside Rust's guarantee [9], and each has a natural form in this subsystem: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation.
- Two further classes are included on purpose from outside that guarantee, a lock-order deadlock and a lost update. They keep the study from confirming a foregone conclusion and show instead where the guarantee stops.
- Scale is what makes the tail matter. One procedure triggers many operations and one site serves many devices, so a p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [6]. This is why the load sweep is pushed past the point where the service starts to fall behind.

4. Objectives

- A documented literature search found no study that measures the tail latency of a Rust and a C++ implementation of the same stateful control-plane subsystem under overload [5].
- Five objectives follow from the research questions: measure predictability (O1), establish which defect classes are prevented (O2), measure the resource overhead (O3), build and measure a migration prototype (O4), and publish reusable artifacts and guidance (O5).
- Three hypotheses are stated in advance, so that the measurements can refute them. H1: below saturation the two implementations perform the same [10]. H2: Rust prevents at compile time the defect classes for which C++ can only offer best-effort detection. H3: the cost of calling Rust from C++ is negligible.
- The work succeeds if the evidence is good enough to base a decision on: repeated runs reported with confidence intervals [11], defect claims shown on programs that really compile and run, migration cost read off a working prototype, and every number reproducible from the published artifacts.
- The scope is drawn narrowly to protect internal validity. The full gNB, the PHY and L2 layers, the user plane, captured traffic traces, managed languages, and production operations are all left out.

5. Research Methodology

- The approach is empirical and constructive. Instead of comparing two systems that already exist and differ in many ways, the study builds a matched pair of implementations from one frozen specification, so that the language is the only systematic difference between them.
- RQ1 and RQ3 are answered with an open-loop load sweep that runs from light load into overload; open-loop means the generator keeps to its send schedule instead of waiting for replies. Latency is recorded in identically configured HDR histograms [8], and resource use is sampled through the operating system.
- RQ2 is answered by controlled fault injection [12]. A fixed catalogue of defects is written in both languages, and the expected outcome of each one is recorded in advance, so that a surprise shows up as a discrepancy instead of being explained away afterwards.
- RQ4 is answered by a working prototype in which C++ drives the Rust core through a foreign-function interface. It yields three measurements: how large the seam is, how much `unsafe` code it needs, and what one call across it costs [13].
- Fairness is the central experimental control: the specification is frozen, allocator and optimisation settings are matched, and every point where the two languages had to differ is written down in a forced-idiom log. Results are reported as medians with bootstrap confidence intervals, together with exact paired permutation tests corrected for multiple comparisons by the Holm procedure [11]. All data is synthetic, so no data-protection concerns arise.

6. Design and Implementation

- A language-neutral specification is frozen before any code is written. It fixes the public API, the state machine, the data structures, the algorithmic strategy, and the sharding and concurrency model, and it serves as the fairness baseline for everything that follows.
- Both versions use the same shared-nothing, thread-per-shard architecture: eight shards, one bounded blocking MPSC queue per shard, single-use completion handles, strict FIFO order within a shard, and a single atomic counter as the only shared mutable state.
- The run-time substrate is equalised as well: the same jemalloc allocator, with the system allocator kept as a control, the same HDR histogram configuration, matching optimisation levels (`-O2` and `opt-level = 2`), and the same hash functions for choosing a shard and for lookups inside it.
- Where the languages must diverge, the forced-idiom log records why. C++ has to hand-write the bounded queue that Rust gets from `sync_channel`, and the tagged request struct and response variant that C++ needs are larger and more branch-heavy than Rust's typed per-variant senders. The result is that the Rust version has more lines but lower total cyclomatic complexity [14].
- The Rust core contains no `unsafe` at all. Equal behaviour is tested rather than assumed: a language-neutral conformance suite drives both implementations, and their canonical transcripts are compared line by line.

7. Experimental Setup and Evaluation

- Two environments have to be kept apart. Everything reported here was measured on a WSL2 development host, which is adequate for parity testing, fault injection, and the structure of the FFI seam, but not for claims about tail latency. The bare-metal protocol is specified and scripted, but it has not been run yet.
- The workload is a deterministic generator of RAN lifecycle traffic, justified by the specification, that follows a fixed open-loop send schedule and is swept from nominal load through saturation. The four variants, two languages by two allocators, are interleaved within every repeat.
- For RQ1 and RQ3, the four variants are indistinguishable at the median and equal in throughput and CPU use [10]. The tail cannot be resolved on this host, because the confidence interval on a single variant's p99 is wider than the spread between all four variants.
- For RQ3, memory is the one performance result that comes out cleanly. The Rust build carries a few MiB more peak resident set under either allocator, the difference has the same sign in every repeat, and it is larger than the difference the allocator makes.
- For RQ2, all five in-scope defect classes are compile errors in safe Rust [9], whereas every one of them compiles in C++ and is caught only if the right sanitiser happens to cover the path. The two boundary classes compile in both languages. For RQ4, a seam of twelve entry points lets C++ drive the Rust core, keeps that core free of `unsafe`, and costs less than a nanosecond per call.

8. Discussion

- Every research question is answered together with the status of its evidence. The safety and migration results are final, because they do not depend on the measurement environment. The performance and resource results are preliminary.
- On performance, the honest reading is that the question about the median is settled and the question about the tail is still open. That is the better position to be in: it leaves the eventual bare-metal measurement free to be a finding rather than a confirmation. The memory penalty is bounded, but explaining it would need hardware counters.
- On safety the difference is one of kind rather than degree, and it has a definite edge. Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors [9].
- On migration, an `unsafe`-free core says what the compiler checks; it says nothing about what a C++ host in the same address space can corrupt [13]. The gain is therefore that the component stops contributing new defects of its own.
- The hypotheses are then revisited: H1 is half answered, H2 holds in its precise form and fails in the loose reading that Rust prevents every concurrency bug, and H3 is supported. The chapter closes with engineering guidelines, related work, threats to validity, and the professional obligation not to over-claim.

9. Conclusion and Future Work

- The thesis compared modern C++ and Rust along four dimensions, on one UE Context Management service built twice from a single frozen specification. Building both versions from one specification separates the effect of the language from differences in design or in the skill of the author.
- Safety: safe Rust rejects all five injected memory-safety and data-race classes at compile time, while the two boundary classes compile and misbehave exactly as they do in C++.
- Migration: an unmodified Rust core can be driven from C++ across a thin seam that confines `unsafe` to a few functions, at less than a nanosecond per call. The real cost of adoption is API design and the learning curve [15].
- Performance: the languages are equivalent at the median and in CPU use and throughput, the Rust build carries a few MiB more peak RSS, and the tail is not yet measurable on a shared host.
- Future work starts with the bare-metal campaign that is already specified, and then extends to production traces, further subsystems, hardware counters, async runtimes, in-process heap isolation, and a longitudinal migration case study.

Bibliography

[1] 3GPP, "Service requirements for the 5G system; stage 1," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 22.261, Mar. 2022, v18.6.0, Release 18. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip

[2] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal war in memory," in *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, 2013, pp. 48–62. [Online]. Available: https://doi.org/10.1109/SP.2013.13

[3] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2nd ed. San Francisco, CA: No Starch Press, 2023.

[4] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others, "The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously," CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Cybersecurity Information Sheet, Dec. 2023. [Online]. Available: https://www.cisa.gov/case-memory-safe-roadmaps

[5] R. Karlbäck and A. Orö, "Holistic view on alternative programming languages for radio access network applications in cloud and embedded deployments," Master's thesis, Linköping University, Department of Computer and Information Science, 2021, carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. [Online]. Available: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307

[6] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. [Online]. Available: https://doi.org/10.1145/2408776.2408794

[7] 3GPP, "NG-RAN; architecture description," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.401, Jun. 2023, v17.5.0, Release 17. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip

[8] G. Tene, "HdrHistogram: A high dynamic range histogram, and the coordinated omission problem," accessed August 2026. [Online]. Available: https://hdrhistogram.org/

[9] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "Safe systems programming in Rust," *Communications of the ACM*, vol. 64, no. 4, pp. 144–152, 2021. [Online]. Available: https://doi.org/10.1145/3418295

[10] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle, "The case for writing network drivers in high-level programming languages," in *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019, pp. 1–13. [Online]. Available: https://doi.org/10.1109/ANCS.2019.8901892

[11] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76. [Online]. Available: https://doi.org/10.1145/1297027.1297033

[12] R. Natella, D. Cotroneo, and H. S. Madeira, "Assessing dependability with software fault injection: A survey," *ACM Computing Surveys*, vol. 48, no. 3, pp. 44:1–44:55, 2016. [Online]. Available: https://doi.org/10.1145/2841425

[13] Z. Li, J. Wang, M. Sun, and J. C. S. Lui, "Detecting cross-language memory management issues in Rust," in *Computer Security – ESORICS 2022*, 2022, pp. 680–700. [Online]. Available: https://doi.org/10.1007/978-3-031-17143-7_33

[14] T. Yin, "Lizard: An extensible cyclomatic complexity analyzer for many programming languages," 2026, version 1.23.0. Accessed August 2026. [Online]. Available: https://github.com/terryyin/lizard

[15] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek, "Benefits and drawbacks of adopting a secure programming language: Rust as a case study," in *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. [Online]. Available: https://www.usenix.org/conference/soups2021/presentation/fulton

Appendix A. Fault-injection example

- This appendix shows one case from the RQ2 catalogue, the use-after-free, in both languages side by side.
- In the C++ version a reference into the context map outlives the `erase` that frees the entry, and the program still compiles cleanly.
- The error appears only at run time, and only if AddressSanitizer is enabled on a path that a test actually exercises.
- The equivalent safe-Rust program does not compile at all: the borrow checker refuses to let the map be modified while a reference into it is still live (`E0499`) [3].
- Side by side, the two programs make the difference in kind of guarantee concrete instead of arguing it from language semantics.